

Samgeet: Supabase setup guide

2026-09-21 · @Someone

Samgeet needs one Supabase project with email sign-in, three database tables and security rules, so a listener can restore their profile and library on any phone. Today everything sits in the phone's local storage, so nothing is backed up and nothing follows you to a new device.

What Samgeet needs from Supabase

Need	Why	Supabase feature
Email one-time-code sign-in	Email is the account key; no password	Auth, Email OTP
Profile row	Name, email, avatar, languages, moods, singers, optional device info	Table <code>profiles</code>
Library backup	Favourites, playlists, history, followed artists, taste, settings	Table <code>library_items</code>
Security rules	Each person can read and write only their own rows	Row Level Security (RLS)
Public playlist links (optional, later)	Replaces the pasted <code>samgeet://p/</code> code	Table <code>shared_playlists</code>
Reliable email delivery	The built-in sender is strictly rate-limited	Custom SMTP

The steps below run in order. Steps 1 to 4 happen in the Supabase dashboard; steps 5 and 6 change the Flutter app; step 7 is the go-live check.

Step 1: Create the project

You finish this step with a project URL and an anon (public) key, the only two values the app needs.

1. Go to supabase.com, sign in with the GitHub account `loco0011`, and choose **New project**.
2. Name it `samgeet`. Pick the region closest to your listeners (for India, Mumbai `ap-south-1`).

3. Set a strong database password and store it in a password manager. The app never uses it.
4. Choose the Free plan and click **Create new project**. Provisioning takes about two minutes.
5. Open **Project Settings → API** and copy two values:
 - **Project URL**, like `https://abcdxyz.supabase.co`
 - **anon public key**, a long string starting `eyJ...`

The anon key is safe to ship inside the app because the security rules in Step 3 do the protecting. Never put the `service_role` key in the app or in GitHub; it bypasses every rule.

Free projects can pause after a week of inactivity and have a small database quota. Check the current limits on Supabase's pricing page before you launch.

Step 2: Create the tables

Three tables cover everything the app stores today. The library is kept as one JSON value per key (`favorites` , `playlists` , `history` , and so on) because that is exactly how `LibraryStore` already saves it on the phone, so syncing is a copy, not a rewrite.

Open **SQL Editor → New query**, paste the script below and click **Run**.

```
-- Who the listener is (mirrors the app's Profile)
create table public.profiles (
  id uuid primary key references auth.users(id) on delete cascade,
  name text not null,
  email text not null,
  avatar text not null default '',
  languages text[] not null default '{}',
  moods text[] not null default '{}',
  artists text[] not null default '{}',
  share_device_info boolean not null default false,
  device jsonb,                -- only filled when share_device_info
is true
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

-- One row per saved key from LibraryStore
create table public.library_items (
  user_id uuid not null references auth.users(id) on delete cascade,
  key text not null check (key in
```

```

('favorites','playlists','history','artists','searches','taste','settings')),
  value jsonb not null,
  updated_at timestamptz not null default now(),
  primary key (user_id, key)
);

-- Optional: public playlist links instead of pasted codes
create table public.shared_playlists (
  code text primary key default substr(md5(random()::text ||
clock_timestamp()::text), 1, 10),
  owner_id uuid not null references auth.users(id) on delete cascade,
  name text not null,
  track_ids text[] not null check (cardinality(track_ids) > 0),
  created_at timestamptz not null default now()
);

-- Keep updated_at honest
create or replace function public.touch_updated_at()
returns trigger language plpgsql as $$
begin new.updated_at = now(); return new; end $$;

create trigger profiles_touch before update on public.profiles
  for each row execute function public.touch_updated_at();
create trigger library_touch before update on public.library_items
  for each row execute function public.touch_updated_at();

```

Check it worked in **Table Editor**: you should see `profiles`, `library_items` and `shared_playlists`, all empty.

Table	One row is	Written by
<code>profiles</code>	One listener	App, after sign-in
<code>library_items</code>	One saved list or setting for one listener	App, after each change
<code>shared_playlists</code>	One public playlist link	App, when you tap Share (optional)

Step 3: Lock the tables down with Row Level Security

The anon key is public, so Row Level Security (RLS) is what stops one listener reading another's data. Turn it on for every table, then allow only the access the app needs. Run this in a new SQL query.

```

alter table public.profiles enable row level security;
alter table public.library_items enable row level security;
alter table public.shared_playlists enable row level security;

-- Each signed-in listener sees and edits only their own profile
create policy "profiles: own row" on public.profiles
  for all to authenticated
  using (id = (select auth.uid()))
  with check (id = (select auth.uid()));

-- ...and only their own library
create policy "library: own rows" on public.library_items
  for all to authenticated
  using (user_id = (select auth.uid()))
  with check (user_id = (select auth.uid()));

-- Shared playlists: owners manage their own links
create policy "shared: owner reads" on public.shared_playlists
  for select to authenticated using (owner_id = (select auth.uid()));
create policy "shared: owner inserts" on public.shared_playlists
  for insert to authenticated with check (owner_id = (select auth.uid()));
create policy "shared: owner deletes" on public.shared_playlists
  for delete to authenticated using (owner_id = (select auth.uid()));

-- Anyone with a code can open ONE playlist, but nobody can list them all
create or replace function public.get_shared_playlist(p_code text)
returns table (name text, track_ids text[])
language sql stable security definer set search_path = public as $$
  select name, track_ids from public.shared_playlists where code = p_code;
$$;
grant execute on function public.get_shared_playlist(text) to anon,
authenticated;

```

The function is why `shared_playlists` has no public read policy: a friend without an account can import a playlist by code, and the list of all codes stays private.

Verify it:

1. In **Table Editor**, each table should show an RLS enabled badge.
2. Open **Advisors → Security**. It should report no "RLS disabled" warnings.
3. After Step 5, sign in as two different test emails and confirm each sees only their own library.

Step 4: Set up email one-time-code sign-in

Listeners will type a 6-digit code sent to their email, with no password to forget. Supabase sends a link by default, so the email templates must be changed to show the code.

1. Open **Authentication → Providers → Email** and confirm it is enabled.
2. Set the email OTP expiry to 600 seconds (10 minutes) and keep the length at 6.
3. Open **Authentication → Email Templates**. Edit both **Confirm signup** (used by first-time listeners) and **Magic Link** (used by returning ones) so they show the code instead of a link:

```
<h2>Your Samgeet code</h2>
<p>Type this code in the app to sign in:</p>
<h1 style="letter-spacing:6px">{{ .Token }}</h1>
<p>It expires in 10 minutes. If you didn't ask for it, ignore this email.</p>
```

4. Set the subject of both templates to `Your Samgeet sign-in code`.
5. Open **Authentication → URL Configuration**. Set **Site URL** to your GitHub repo page for now and add `samgeet://auth` under **Redirect URLs**. Codes do not use them, but this keeps a later magic-link option working.

Use your own email sender

Supabase's built-in sender is only meant for testing: it is strictly rate-limited and only delivers to your own team's addresses. Before real listeners sign in, connect an SMTP provider:

1. Create an account with an email provider such as Resend, Brevo or Postmark, and verify a sender address (a domain you own gives the best delivery).
2. Open **Authentication → Emails → SMTP Settings** and switch on **Enable custom SMTP**.
3. Enter the sender name (`Samgeet`), sender address, host, port, username and password from the provider.
4. Save, then request a code for your own email and confirm it arrives in the inbox, not spam.

Also review **Authentication → Rate Limits** and lower the email limit to a value you are comfortable with, so nobody can use your sender to flood an inbox.

Step 5: Connect the Flutter app

The app talks to Supabase through the `supabase_flutter` package, with the URL and anon key passed in at build time so they stay out of the source code.

1. Add the package: `flutter pub add supabase_flutter`.
2. Start Supabase in `lib/main.dart`, before `runApp`:

```
const _url = String.fromEnvironment('SUPABASE_URL');
const _anon = String.fromEnvironment('SUPABASE_ANON_KEY');

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();
  if (_url.isNotEmpty && _anon.isNotEmpty) {
    await Supabase.initialize(url: _url, anonKey: _anon);
  }
  // ...existing startup...
}
```

When the two values are missing the app skips Supabase and keeps working as it does today, on-device only.

3. Send and check the code. This replaces the local-only sign-in in `sign_in_screen.dart`:

```
final auth = Supabase.instance.client.auth;

// screen 1: the listener types their email
await auth.signInWithOtp(email: email, shouldCreateUser: true);

// screen 2: they type the 6-digit code from the email
final res = await auth.verifyOTP(email: email, token: code, type:
OtpType.email);
final user = res.user; // null means a wrong or expired code
```

4. Build with the keys from Step 1:

```
flutter run --dart-define=SUPABASE_URL=https://abcdxyz.supabase.co --dart-
define=SUPABASE_ANON_KEY=eyJ...
flutter build apk --release --dart-define=SUPABASE_URL=... --dart-
define=SUPABASE_ANON_KEY=...
```

Sign-out becomes `auth.signOut()` plus clearing the local copy, as `LibraryStore.signOut()` does today. The Android manifest already allows internet access, so no permission changes are needed.

File	Change
<code>pubspec.yaml</code>	Add <code>supabase_flutter</code>
<code>lib/main.dart</code>	Initialise Supabase from the build-time keys

File	Change
<code>lib/data/cloud_service.dart</code> (new)	Wraps sign-in, profile and library upload and download
<code>lib/ui/screens/sign_in_screen.dart</code>	Add the code-entry step after the email
<code>lib/data/library_store.dart</code>	Call the cloud service after each saved change

Step 6: Sync and restore data

The phone stays the working copy, so the app still runs offline; Supabase holds the backup and hands it to a new phone. This is the part with the most app code, so build it after Steps 1 to 5 work.

Upload. `LibraryStore` already batches saves in `flush()` after 500 ms. After each key is written locally, upsert the same JSON to Supabase:

```
await client.from('library_items').upsert({
  'user_id': client.auth.currentUser!.id,
  'key': key, // 'favorites', 'playlists', ...
  'value': jsonValue,
}, onConflict: 'user_id,key');
```

Save the profile the same way into `profiles`, using the signed-in user's id as `id`. If the phone is offline, skip the upload and retry on the next change or app start.

Restore. Right after a successful code check, download the listener's rows and combine them with what is on the phone:

Data	If the account already has it and the phone does too
Favourites, history, followed artists	Union by song or artist id; history capped at 150
Playlists	Union by playlist id; if the id matches, keep the one with more songs
Taste and settings	Keep the copy with the newer <code>updated_at</code>
Profile	Account copy wins; fill blanks from the phone

A first-time listener has nothing on the server, so their existing on-device profile and library simply upload. That is how people already using version 1.1.0 keep their data.

Delete account. Google Play requires an in-app way to delete an account. The app cannot delete an auth user itself, so add a small Edge Function that calls `auth.admin.deleteUser` with the service role key kept as a function secret. Because every table uses `on delete cascade`, the profile and library rows disappear with the user.

Open question: should signing out keep the library on the phone or clear it? Keeping it is friendlier on a personal phone; clearing it is safer on a shared one.

Step 7: Test and go live

Do not release until every box below is ticked.

Security

- ☐ RLS is on for all three tables and Advisors → Security shows no warnings
- ☐ Two test accounts cannot see each other's profile or library
- ☐ The `service_role` key appears nowhere in the app, the repo or a build command
- ☐ Only the anon key is passed with `--dart-define`
- ☐ Custom SMTP is on and a code reaches a Gmail inbox, not spam

Behaviour

- ☐ A new email gets a code, and a wrong or expired code shows a clear error
- ☐ Signing in on a second phone restores favourites, playlists and profile
- ☐ The app still opens and plays music in airplane mode
- ☐ A build without the Supabase keys still runs as a guest
- ☐ Delete account removes the profile and library rows

Housekeeping

- ☐ Update the README and the About screen: data is now stored in your Supabase account, not only on the phone
- ☐ Update the privacy wording, since the app currently says it uploads nothing
- ☐ Keep the database password in a password manager, and turn on backups if you move to a paid plan
- ☐ Bump the version, rebuild the release APK with the keys, and attach it to a new GitHub release